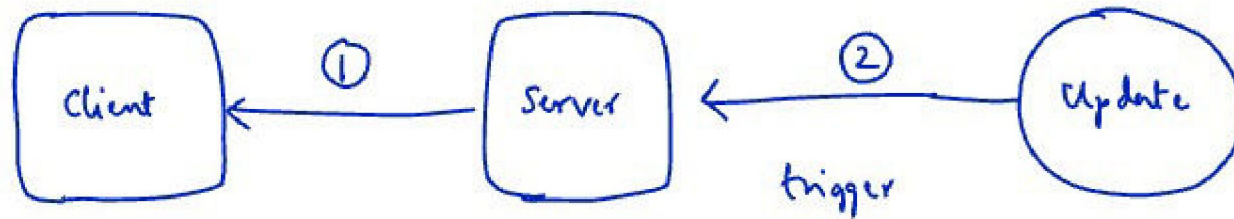


- Realtime updates :

Eg: Notifications

Stock market data

Chat messages etc



1. Server to Client
2. Updates to Server

Simple Polling with a certain interval.
(2-5s)

long polling

SSE

Websockets

WebRTC

- Simple polling

- Pros

- easy to implement
 - Stateless
 - No special infrastructure
 - Easy to explain

- Cons

- Overhead + higher bandwidth
 - Higher latency

- `setInterval( callback, 2000 )`

↓
make
request

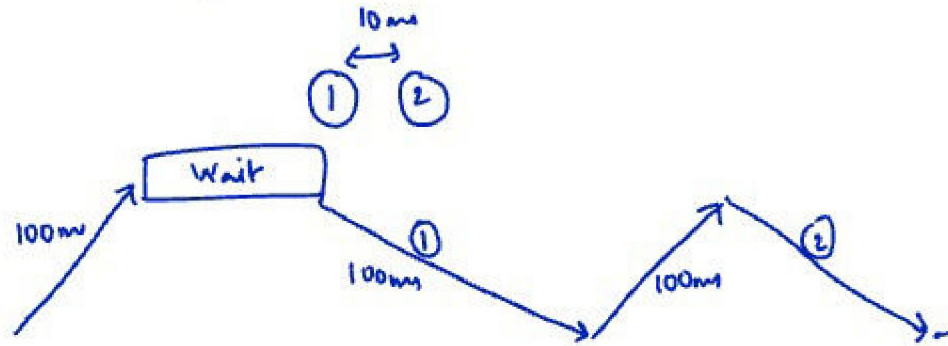
↓
2s

↓
discuss polling frequency

- keep alive - to make it more efficient

- long polling

- Server returns any new events if available or waits until there is an event or timeout



time gap for update 1: 100ms
(between occurrence & client receiving)

time gap for update 2: $90 + 100 + 100 = 290ms$

- Pros:

- Built on HTTP
- Easy to implement
- Stateless

- Cons:

- latency
- Higher overhead than other alternatives

- Monitoring gets skewed - time taken for requests false timeout events registered
- Browsers have limit on #TCP connections (6)

- good choice when updates are infrequent + simple solution preferred

- useful: for apps with a long async process where we want to immediately know when it's done

Eg: payment confirmation

discuss

- polling frequency (duration):
 - each hop in the infra needs to know about the long polling

• SSE

- Transfer-Encoding: chunked

└ indication to client to not wait for the whole response before parsing

→ can parse each event as it's received.

- Most of events are real time
- Browsers have support - EventSource

↓
reconnecting is taken care of by this ⇒ no extra code

A - Atomic - all or nothing

C - Consistency

I - Isolation

D - Durability } Persistence

- VSE cases

• AIE cases: tokens streaming
(all tokens are gradually generated one after other)

Pros:

- Built into browsers
- on top of HTTP
- Automatic reconnection - EventSource
- lower latency
- simple to implement

Cons:

- One way communication
- Modern browsers
- Browsers limiting #TCP (concurrent) connections
- Same monitoring issue
- Stateless

- Websockets: High frequency, bidirectional updates
 ↓
 (important)
 reads & writes
- Build on HTTP - upgrade protocol
 that lets the TCP connection
 change protocol at layer 7
- Every piece of infrastructure needs to support
 (layer 7) web sockets
- Exchange any kind of message: JSON, binary,
 images, protobufs
- Connection stays until explicitly closed.
 - very long lived

- Extra challenges
 - L7 load balancer doesn't guarantee
 connecting the user to the same server
 → use L4 LBs for Websockets
 - deployments:
 - sever old connections
 & reconnect server are up
 - mechanism for handling missed
 updates

- Load balancing:
 - least connections strategy
 - stickiness

- Separate Websockets service & a separate application server
 - redeployed rarely
 - redeployed frequently
 & remain stateless
 → scaled
 independently

- Pros:
 - Full duplex (read & write)
 - lower latency
 - Efficient for high frequency
 - wide browser support

- Cons:
 - complex (relatively)
 - infrastructure should support it
 - stateful (scaling trickier)
 - handle reconnection